

Python 2: Visualization and Analysis - Assignments

Python 2 – Part 1 (Data Visualization Using matplotlib and plotly)

Exercise 1 – Time Series

In this exercise you will practice plotting time series data, annotating and labeling graphs with either matplotlib or plotly.

1. Load the Apple stock data stored in AAPL.csv into a pandas data frame.
 - a. Set the index to be DateTime type.
2. Plot the 'Adj Close' as a time series plot using matplotlib .plot() or plotly .line().
3. Label the title along with the X and Y axis.
4. Find the global minimum and maximum. Store these values in variables so that you can annotate them on the graph.
 - a. Use .min() and .max() methods to find the values and .idxmin() / .idxmax() to find the location (date).
 - b. Annotate the max/min prices using matplotlib .annotate() or plotly .add_annotation().

Challenge: Find and annotate the min and max for each year. This can be done in a loop.

Exercise 2 – Multiple Graphs

In this exercise you will practice creating multiple plots using subplots.

1. Load the stock data for the following tickers: S&P500, AAPL, AXP, BA, CAT, CSCO.
 - a. Set the index to be DateTime type.
2. Create a subplot with 3 rows and 2 columns.
3. Plot the 'Adj Close' for each ticker.
 - a. Remember to set the title and label the y-axis of each graph.

Challenge: Use loops to automate the process to load the data, concatenate into a single data frame. Below are some code snippets and hints to help.

```
# create a list of all csv files
import os
stockdatalist = []
for file in os.listdir("../StockData/"):
    if file.endswith(".csv"):
        stockdatalist.append(file)

dfs = [] #empty list to store all loaded csv files
# Create a loop that iterates over the list of csv files
# Inside the loop, load the csv to a dataframe, and append the dataframe to the dfs list.
```

Python 2: Visualization and Analysis - Assignments

Exercise 3 – Box Plots & Histograms

In this exercise you will practice creating box plots and histograms with plotly.

1. Load the stock data for the following tickers: S&P500, AAPL, AXP, BA, CAT, CSCO. Hint: you can reuse the same code from exercise 2.
2. Calculate the daily returns for each of the tickers and merge all the tables into one DataFrame.
3. Use plotly's `.box()` function to create box plots of the returns for all the tickers.
4. Use plotly's `.histogram()` function to create multiple histograms of the returns of all tickers
 - a. Experiment with adding the function argument `facet_col` or `facet_row` to show the histograms side by side or on top of each other.

Exercise 4 – Histograms

In this exercise you will practice creating histograms, and overlaying line graphs.

1. Load the S&P500 data into a data frame.
2. Calculate the log returns, drop NAs, and assign the values into a numpy array.
 - a. This can be done in a single line stacking the `.diff().dropna().values` methods.
3. Find the mean and standard deviation of returns and store them in variables named `mu500` and `std500` respectively.
4. Using the `np.linspace()` function create a range of 100 numbers that starts 3 standard deviations below and end 3 standard deviations above the mean return.
5. Plot the histogram of returns using 100 bins.
6. Overlay the normal pdf.
 - a. Using `plt.plot()` pass the linear range as the x-axis, and the y-axis will be the normal pdf evaluated using the `norm` function in the `scipy.stats` package as follows: `norm.pdf(norm500, mu500, std500)` (import using the following command from `scipy.stats` `import norm`).
7. Create a textbox with the mean and standard deviation displayed to 3 decimal places.
8. Show the plot.